Winning Space Race with Data Science

Gunter Nembach
15.05.2026

Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

- Summary of methodologies
    - Request data via SpaceX API & Web Scraping
    - Clean and normalize data using Python
    - EDA (exploratory data analysis): data wrangling/ missing values with Python & SQL
    - Visual Data analysis with Python/ Plotly/ Folio

Executive Summary

- Summary of all results
  - Falcon 9: Launches for each site
    - Cape Canaveral, SLC 40: 55
    - Kennedy, LC-39A: 22
    - Vandenberg, SLC-4E: 13
  - Falcon 9: Top 5 Orbits*
    - ISS (21), VLEO(14), PO(9), LEO(7),SSO(5)
  - Falcon 9: Top 3 Mission outcomes
    - Landing Droneship: 41
    - Failure Landing: 19
    - Landing Ground Pad: 14

* see appendix for orbit overview

Executive Summary

- Summary of all results
  - Higher Flight Numbers -> higher Payload Mass/ success Rate
  - No Payload Mass > 10t from Vandenberg 4E
  - 100% Success Rate - Orbits: ES-L1 / GEO/ HEO/ SSO*
  - Target Orbits with Payload >8t: ISS/ PO/ VLEO
  - Avoid proximities to railway/ cities/ highways -> safety
  - Positiv proximities to ocean/ equator -> safety / energy consumption

Introduction

- Project background and context

  SpaceX operates Starlink satellite internet and significantly reduces launch costs through reusable Falcon 9 rockets (about $62 million vs. over $165 million). A key factor is whether the first stage successfully lands and can be reused, since it performs most of the work and represents a major portion of the cost.

- Problems you want to find answers

  As data scientists at a competing company SpaceY we use public data and machine learning to predict whether the first stage will land and be reused in order to estimate launch costs and be competitive to SpaceX.

Section 1

Methodology

Methodology

Executive Summary

- Data collection methodology:
  - Data collected via SpaceX API and Web Scraping
- Perform data wrangling
  - Data was transferred to Panda database and missing values were cleaned
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models
  - Dividing the dataset into test- and training data, using different classification models like KNN/ Logistic Regression/ … and tuning Hyperparameters with GridSearchCV

Data Collection

- The data are collected via
  - SpaceX API
    "https://api.spacexdata.com/v4/rockets/"
  - Webscraping
    “https://en.wikipedia.org/w/index.php?title=List_of_Falcon_9_and_Falcon_Heavy_launches&oldid=1027686922”
- You need to present your data collection process use key phrases and flowcharts

Data Collection – SpaceX API

The notebook collects SpaceX launch data through REST calls, enriches each launch with rocket, payload, launchpad, and core metadata, and then cleans the result into one analysis-ready table. This makes landing prediction possible because the final dataset contains the key explanatory features needed for modeling

Key Phrases

- **GET request**: Abruf der SpaceX-Launchdaten über die REST API.
- **JSON normalization**: Verschachtelte API-Antworten werden in tabellarische Form überführt.
- **Helper functions**: Anreicherung der Launches mit Rocket-, Payload-, Launchpad- und Core-Metadaten.
- **Data wrangling**: Entfernen von Multi-Core-/Multi-Payload-Fällen und Umwandlung von Datumswerten.
- **Final dataset**: Ein bereinigter Datensatz pro Launch als Grundlage für Analyse und Modellierung.

.

- GitHub URL:

https://github.com/gnembac/IBM_Datascience_Capstone_PRJ/blob/a8c54efc091b1b1d221046e2634f1f7156b684ec/jupyter-labs-spacex-data-collection-api%20(3).ipynb

```
flowchart TD

A[Start] --> B[GET /v4/launches/past]

B --> C[Load JSON into dataframe]

C --> D[Extract rocket ids]

C --> E[Extract payload ids]

C --> F[Extract launchpad ids]

C --> G[Extract core ids]

D --> H[GET /v4/rockets/{id}]

E --> I[GET /v4/payloads/{id}]

F --> J[GET /v4/launchpads/{id}]

G --> K[GET /v4/cores/{id}]

H --> L[Append booster version]

I --> M[Append payload mass/orbit]

J --> N[Append site name/lat/lon]

K --> O[Append landing and reuse features]

L --> P[Merge into master table]

M --> P

N --> P

O --> P

P --> Q[Clean rows and types]

Q --> R[Final analysis dataset]
   R --> S[End]
```

Data Collection - Scraping

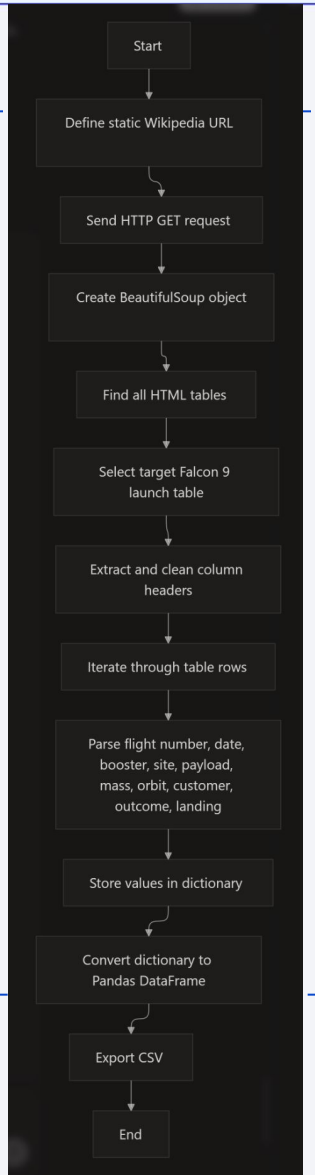
The web scraping pipeline starts with a fixed Wikipedia snapshot, parses the HTML with BeautifulSoup, extracts the Falcon 9 launch table, cleans the headers, and loops through each row to build a structured launch dataset. The resulting dataframe is then exported as CSV for later analysis and modeling

Key Phrases

- **HTTP GET request**: Abruf der Wikipedia-Seite über eine feste Snapshot-URL.
- **BeautifulSoup parsing**: Umwandlung des HTML-Dokuments in eine durchsuchbare Struktur.
- **Target table selection**: Auswahl der relevanten Launch-Record-Tabelle aus allen HTML-Tabellen.
- **Header cleaning**: Bereinigung der `<th>`-Spaltennamen zu lesbaren Feldnamen.
- **Row parsing**: Extraktion und Normalisierung der Launch-Daten aus jeder Tabellenzeile.
- **DataFrame creation**: Aufbau eines strukturierten Pandas-DataFrames.
- **CSV export**: Speichern des bereinigten Datensatzes für die spätere Analyse.

GitHub URL:

https://github.com/gnembac/IBM_Datascience_Capstone_PRJ/blob/c72eb5257a51299c0e52dbd05b9c95d2704f60d0/jupyter-labs-webscraping%20(2).ipynb



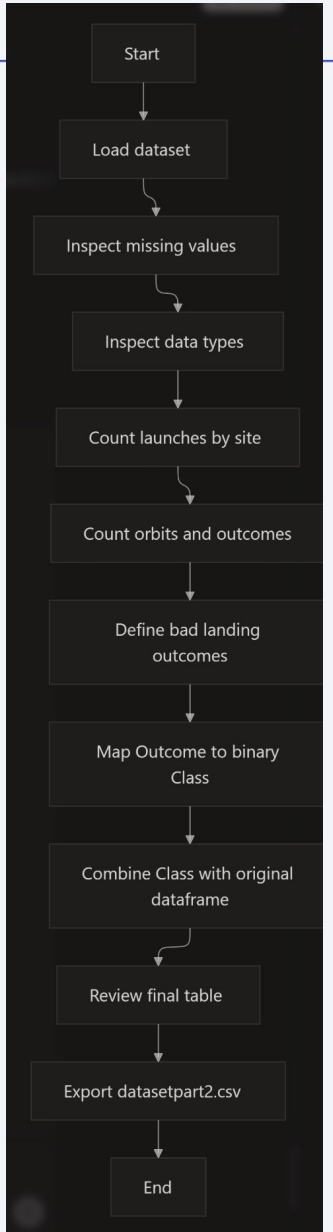
Data Wrangling

The wrangling step starts with the merged SpaceX launch table, then checks missing values and variable types before analyzing launch sites, orbits, and landing outcomes. After identifying the unsuccessful landing patterns, the workflow converts the Outcome column into a binary Class label and exports the final modeling dataset.

Key Phrases

- **Load cleaned dataset**: Start mit der strukturierten Launch-Tabelle aus der vorherigen Phase.
- **Missing-value check**: Prüfung fehlender Werte pro Attribut.
- **Type inspection**: Trennung numerischer und kategorialer Variablen.
- **Launch count analysis**: Anzahl der Starts pro Launch Site.
- **Orbit analysis**: Häufigkeit der Orbit-Typen und Mission Outcomes.
- **Outcome mapping**: Umwandlung der Outcome-Werte in Success/Failure-Klassen.
- **Label creation**: Binäre Zielvariable `Class` mit `1 = success` und `0 = failure`.
- **Feature-ready table**: Export des finalen Modellierungsdatensatzes als CSV.

GitHub URL
https://github.com/gnembac/IBM_Datascience_Capstone_PRJ/blob/d7b4d9271b0fce764147d878e0a3bf6d99c6e058/labs-jupyter-spacex-Data%20wrangling%20(3).ipynb



EDA with Data Visualization

The notebook used scatter plots to explore how launch success relates to flight number, launch site, orbit type, and payload mass, and it used a bar chart to compare success rates across orbit types. Scatter plots were best for showing patterns and clustering, while the bar chart was best for comparing categorical success rates.

The notebook plots a small set of EDA charts centered on launch success, orbit, payload mass, and launch site behavior. The main goal was to reveal patterns that help explain when Falcon 9 first-stage landings succeed or fail.

Charts plotted

- **Scatter plot: Flight Number vs. Launch Site, colored by Class.** This was used to see whether launch outcomes changed over time at each site and whether success clustered by site.
- **Bar chart: Success Rate by Orbit Type.** This was used to compare the average success rate across orbit categories, which is a simple way to rank orbit types by landing outcome.
- **Scatter plot: Flight Number vs. Orbit Type.** This was used to inspect whether later flights were associated with different orbit patterns and whether some orbit types showed more successful launches as flight number increased.
- **Scatter plot: Payload Mass vs. Orbit Type.** This was used to check whether payload weight was linked to specific orbit choices and whether payload mass appeared related to landing success.

Why these charts

- **Scatter plots** were chosen for relationships between one numeric variable and one categorical variable because they make trends, clustering, and separation between success classes easy to see.
- **Bar charts** were chosen for orbit success rate because the task was to compare categories directly, and a bar chart makes relative performance easy to read.
- **Color coding by Class** was used to separate success and failure visually, which makes landing patterns more interpretable without adding extra tables.

What the visuals showed

- The **Flight Number vs. Launch Site** chart was used to compare site behavior over time and to spot differences between launch locations.
- The **success-rate bar chart** highlighted which orbit types were more favorable for successful landings.
- The **Flight Number vs. Orbit** plot suggested that some orbit types, especially LEO, showed a clearer relationship with increasing flight number than others.
- The **Payload Mass vs. Orbit** plot helped show how payload constraints differ by orbit, including the observation that very heavy payloads were not launched from VAFB-SLC in the notebook's analysis.

GitHub URLhttps://github.com/gnembac/IBM_Datascience_Capstone_PRJ/blob/43a036964402ff6f4bec78a0f9acdd419aeee439/edadataviz.ipynb

EDA with SQL

- **Loaded the SpaceX dataset into SQLite.** I imported the CSV into a table called `SPACEXTBL`, then created a cleaned working table `SPACEXTABLE` after removing blank rows.
- **Listed all unique launch sites.** I used `SELECT DISTINCT LaunchSite FROM SPACEXTABLE` to identify the available launch locations.
- **Filtered launch sites by string pattern.** I queried launch sites beginning with `CCA` using a `LIKE` condition to return the CCAFS-related sites.
- **Computed total payload mass for NASA CRS missions.** I used `SUM(PAYLOADMASSKG)` with a customer filter to measure the total payload carried for NASA CRS launches.
- **Selected payload details for NASA CRS records.** I retrieved payload mass, booster version, and customer fields to inspect individual NASA CRS launches more closely.
- **Calculated average payload mass for booster version F9 v1.1.** I used `AVG(PAYLOADMASSKG)` with a booster-version filter to summarize typical payload weight for that rocket version.
- **Found the first successful ground pad landing date.** I used `MIN(Date)` filtered by `LandingOutcome = 'Success ground pad'` to identify when that milestone first occurred.
- **Listed boosters with successful drone ship landings and mid-range payloads.** I filtered by `LandingOutcome = 'Success drone ship'` and `PAYLOADMASSKG BETWEEN 4000 AND 6000` to find matching boosters.
- **Counted successful vs. failed mission outcomes.** I used conditional counting with `CASE WHEN` expressions to compare overall success and failure totals.
- **Found boosters carrying the maximum payload.** I used a subquery with `MAX(PAYLOADMASSKG)` to return every booster version tied to the heaviest payload.
- **Extracted 2015 drone-ship failure records.** I used date substring logic to pull month-based records for 2015 where the landing outcome was a drone-ship failure.
- **Ranked landing outcomes by frequency.** I grouped by `LandingOutcome`, counted occurrences, and sorted descending to see which landing types appeared most often between 2010-06-04 and 2017-03-20.

GitHub URL: https://github.com/gnembac/IBM_Datascience_Capstone_PRJ/blob/4fc42e1cc0894dbc19d0144c427e8c40f5a64434/jupyter-labs-eda-sql-coursera_sqllite%20(4).ipynb

Build an Interactive Map with Folium

- **Map base layer:** I created a `folium.Map` centered on the NASA Johnson Space Center as the starting view for the launch-site analysis. This gave a geographic reference point for all later objects on the map.
- **Circle objects:** I added `folium.Circle` objects for each launch site, with a fixed radius and popup label showing the site name. The circles were used to make the launch locations easy to spot and to provide a consistent visual footprint for each site.
- **Text-labeled markers:** I added `folium.Marker` objects using `DivIcon` text labels such as `NASA JSC`, `CCAFS LC-40`, `CCAFS SLC-40`, `KSC LC-39A`, and `VAFB SLC-4E`. These labels made the map readable at a glance without needing to click each point.
- **Success/failure markers:** I added color-coded `folium.Marker` objects for launch records, with green for success and red for failure. This was done so the launch outcome could be recognized immediately from the marker color.
- **Marker clusters:** I used `folium.plugins.MarkerCluster` to group many nearby success/failure points. Clustering keeps the map usable when there are many launches in the same area and prevents visual overlap.
- **Mouse position control:** I added `folium.plugins.MousePosition` so coordinates appear while moving the cursor over the map. This helped me identify exact points of interest such as coastline, railway, highway, or city locations.
- **Distance lines:** I created `folium.PolyLine` objects from each launch site to a nearby proximity point, such as coastline or another landmark. The lines were added to visualize distance relationships directly on the map instead of only listing numeric values.
- **Distance labels:** I used a marker with a `DivIcon` to display the computed distance, such as a distance in km. This made the map self-explanatory by tying the line to a numeric distance value.

Why these objects

- Circles and labels were used for **site identification** and to make launch locations easy to compare spatially.
- Colored markers were used for **success classification** so launch outcomes could be interpreted visually.
- Clusters were used for **density handling** when many launches occupied the same region.
- Mouse position and distance lines were used for **proximity analysis** to explore how close launch sites are to coastlines, highways, railways, or cities.

Git Hub URL:https://github.com/gnembac/IBM_Datascience_Capstone_PRJ/blob/5dfcd4ff9156e9b0c30787707afdd3cd3507c631/lab_jupyter_launch_site_location.ipynb

Build a Dashboard with Plotly Dash

- **Pie chart for launch success by site:** I added a pie chart that shows the distribution of successful launches across all launch sites when “All Sites” is selected. This gives a quick high-level view of which sites contributed most to successful missions.
- **Pie chart for success vs. failure at one site:** When a specific launch site is selected, the pie chart switches to a success/failure split for that site. I added this because it makes site-level performance easier to compare than using raw counts alone.
- **Scatter chart for payload vs. outcome:** I added a scatter plot with payload mass on the x-axis and launch outcome on the y-axis. This was included to check whether payload size appears related to launch success or failure.
- **Color grouping by booster category:** The scatter plot colors points by booster version category. I used this to make it easier to see whether different booster generations behave differently across payload ranges.

Interactions

- **Launch site dropdown:** I added a dropdown that lets the user switch between “All Sites” and individual launch sites. This interaction makes the dashboard exploratory rather than static.
- **Payload range slider:** I added a range slider to filter launches by payload mass. This helps users focus on a specific payload interval and see whether success patterns change with weight.
- **Dynamic callbacks:** Both charts update automatically when the dropdown or slider changes. I added callbacks so the dashboard responds instantly without reloading the page.
- **Hover information:** The scatter plot includes hover data such as launch site, payload mass, and outcome. I added this so users can inspect individual launches without cluttering the chart.

Why these choices

- The **pie charts** are best for categorical proportion questions, such as “how much success came from each site” or “what is the success/failure mix at one site.”
- The **scatter plot** is best for seeing relationships between payload mass and mission outcome, especially when combined with booster category coloring.
- The **dropdown and slider** make the dashboard interactive and support drill-down analysis, which is useful for comparing launch sites and payload ranges side by side

Git Hub URL:https://github.com/gnembac/IBM_Datascience_Capstone_PRJ/blob/8a1ffe090f7609bce76c20859dbf3591691bb5ca/spacex-dash-app%20(2).py

Predictive Analysis (Classification)

Process summary

- **Feature preparation:** Created the target label `Class`, standardized the predictors with `StandardScaler`, and split the data into train/test sets with an 80/20 split.
- **Baseline modeling:** Trained and tuned four classifiers: Logistic Regression, Support Vector Machine, Decision Tree, and K-Nearest Neighbors.
- **Hyperparameter search:** Used `GridSearchCV` with 10-fold cross-validation to tune each model and select the best parameter combination.
- **Validation scoring:** Compared models using cross-validation accuracy and then checked test-set accuracy for the fitted best estimator.
- **Error inspection:** Plotted confusion matrices to inspect misclassifications, especially false positives and false negatives.
- **Model selection:** Chose the model with the highest test performance, which was the **Decision Tree classifier**.

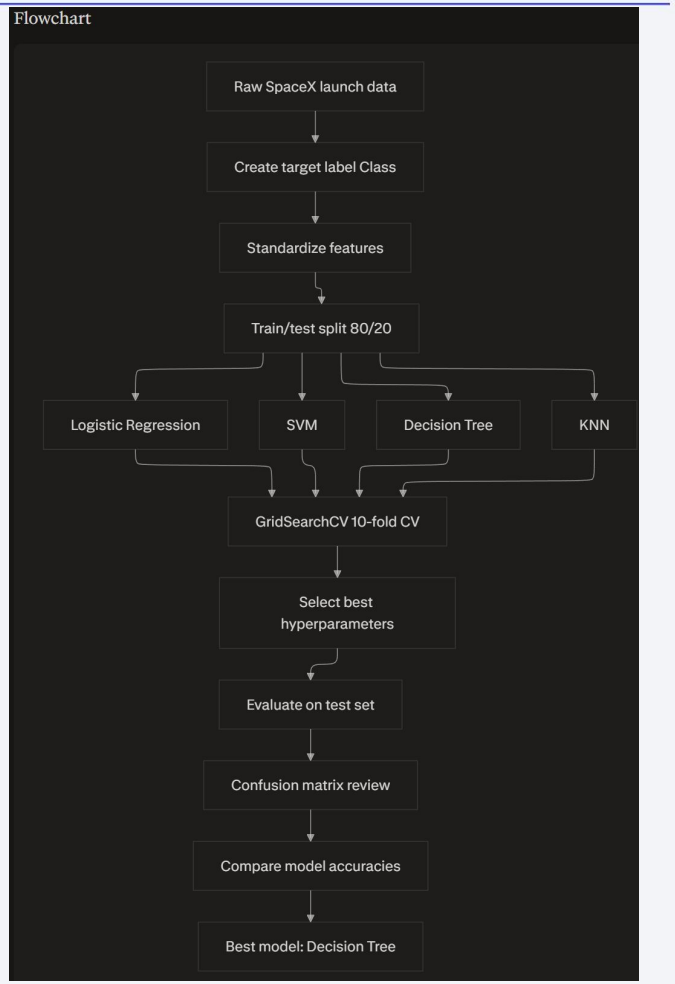
Best model results

- **Logistic Regression:** Best CV accuracy about 0.8464, test accuracy about 0.8333.
- **SVM:** Best CV accuracy about 0.8482, with best parameters `C=1.0`, `gamma=0.03162277660168379`, `kernel='sigmoid'`.
- **KNN:** Best CV accuracy about 0.8482, with best parameters `algorithm='auto'`, `n_neighbors=10`, `p=1`.
- **Decision Tree:** Best CV accuracy about 0.8893, with best parameters `criterion='entropy'`, `max_depth=16`, `max_features='sqrt'`, `min_samples_leaf=2`, `min_samples_split=2`, `splitter='random'`.
- **Winner:** The notebook explicitly states that the **decision tree has the best accuracy**.

Why this process

- Standardization was needed because the feature set contains mixed scales, and scale-sensitive models like SVM and KNN benefit from it.
- Grid search with cross-validation was used to reduce overfitting and make hyperparameter selection more reliable.
- Confusion matrices were added because accuracy alone can hide asymmetric errors, especially false positives in landing prediction.

Git Hub URL:
https://github.com/gnembac/IBM_Datascience_Capstone_PRJ/blob/6e21daf03c41eb1c9b2a88be2b37c3e93329804c/SpaceX_Machine%20Learning%20Prediction_Part_5.ipynb



Results

Exploratory analysis

- The EDA results showed that **launch success varies by site, orbit, and payload mass**, which justified deeper drill-down analysis rather than treating all launches as one population.
- The notebook used scatter plots and bar charts to reveal patterns in launch site behavior, orbit-specific success rates, and payload distribution across missions.
- The SQL notebook reinforced this by summarizing key mission facts such as unique launch sites, payload totals, average payloads, first successful ground-pad landing date, and landing outcome frequencies.

Interactive demo

- The interactive Folium demo mapped launch sites with circles, labeled markers, and outcome markers so the viewer could inspect **where** launches happened and how success/failure clustered geographically.
- The dashboard added a dropdown, a payload range slider, a success pie chart, and a payload-vs-outcome scatter plot to support **drill-down exploration** by launch site and payload range.
- These interactions were designed to make the analysis exploratory rather than static, letting the user change the slice of data instantly.

Predictive results

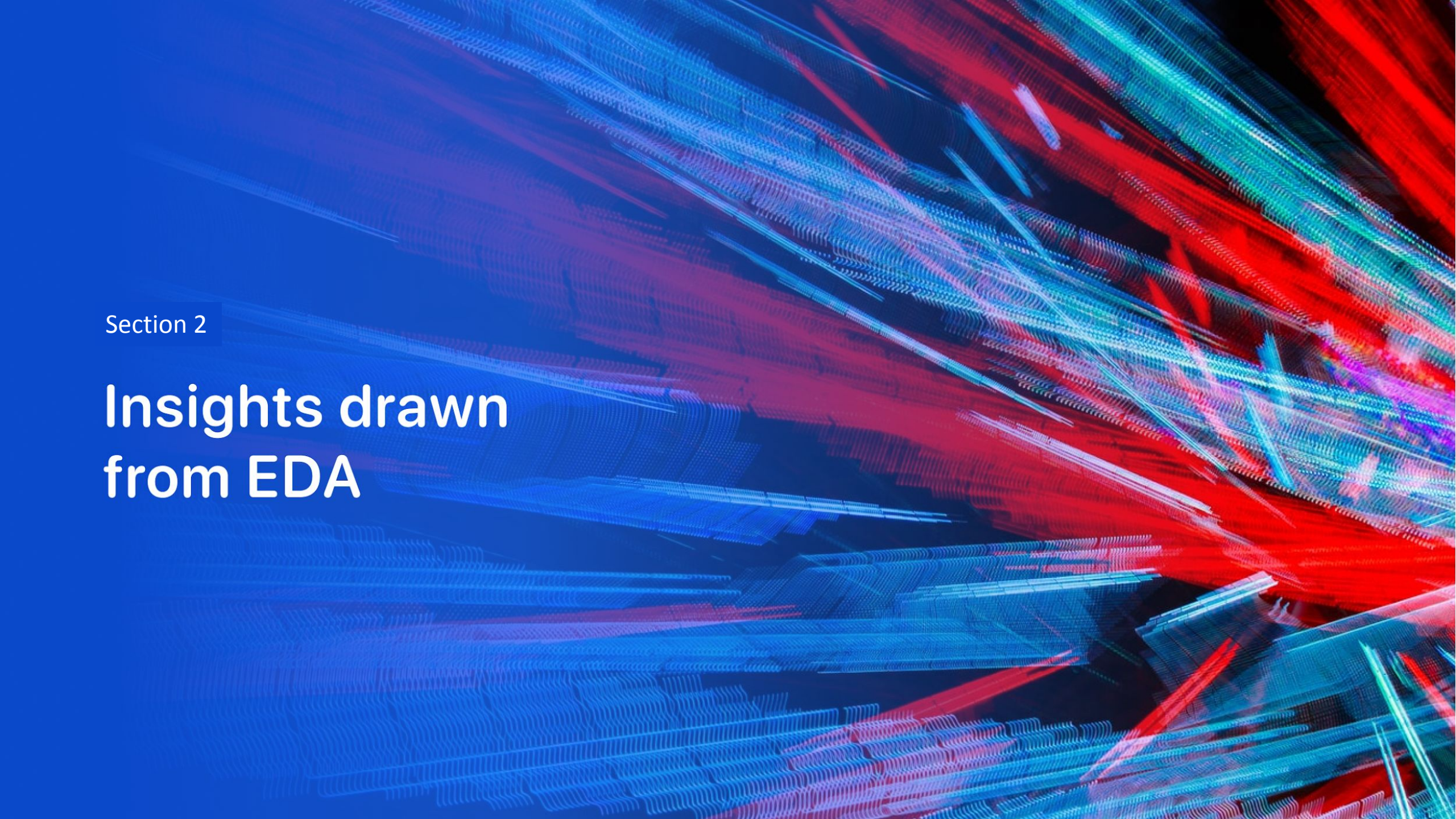
- The machine-learning notebook standardized the features, split the data into training and test sets, and tuned Logistic Regression, SVM, Decision Tree, and KNN models using 10-fold cross-validation.
- The best-performing classifier was the **Decision Tree**, with validation accuracy about 0.8893 and the best tuned parameters `criterion=entropy`, `max_depth=16`, `max_features=sqrt`, `min_samples_leaf=2`, `min_samples_split=2`, and `splitter=random`.
- The notebook compared models on test data and used confusion matrices to inspect misclassification patterns, especially false positives.

Presentation-ready version

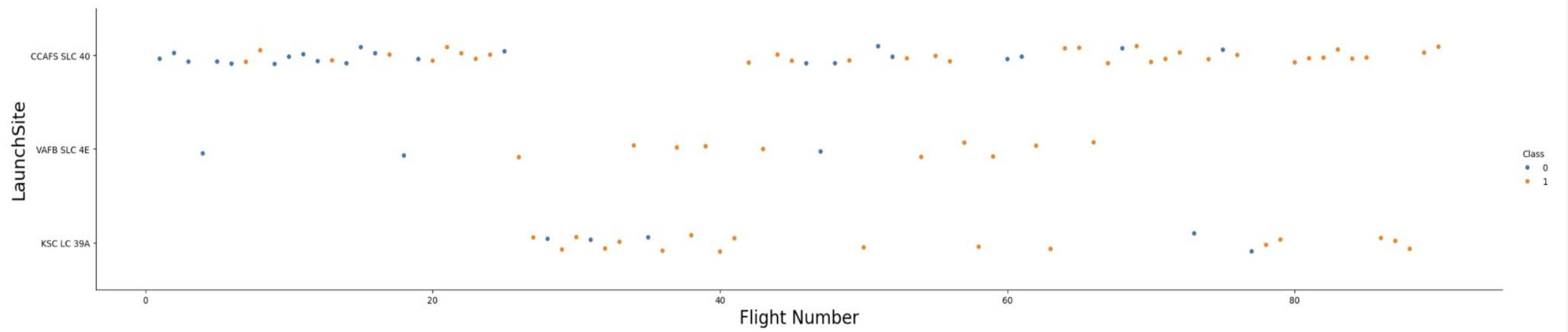
- **Exploratory data analysis results:** launch success depends on launch site, orbit type, and payload mass; unique launch sites and landing outcome counts confirm strong structure in the data.
- **Interactive analytics demo in screenshots:** Folium maps, labeled launch sites, success/failure markers, and a Dash dashboard with dropdowns and sliders show the data interactively.
- **Predictive analysis results:** after scaling and hyperparameter tuning, the decision tree was the best classifier among the tested models.

Section 2

Insights drawn from EDA



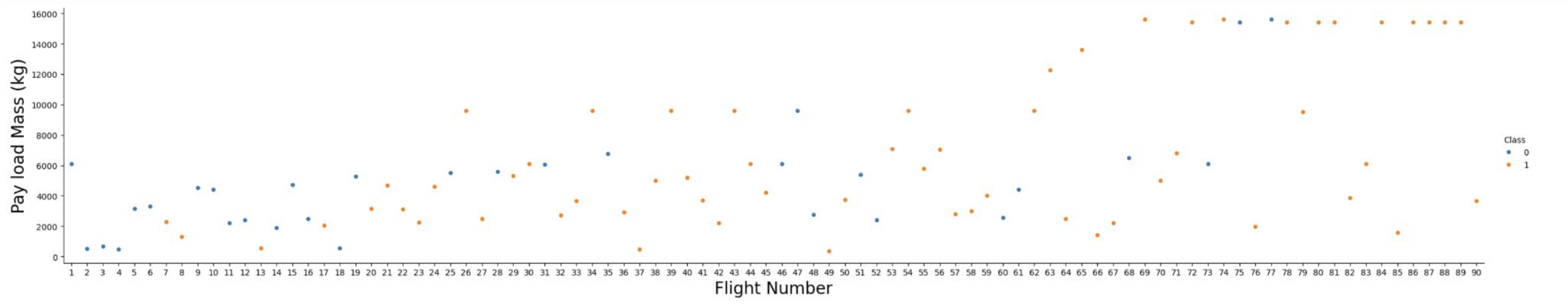
Flight Number vs. Launch Site



- Cape Canaveral, SLC 40
- Kennedy, LC-39A
- Vandenberg, SLC-4E

Higher Flight Number means more lessons learned and improvements for safe flights

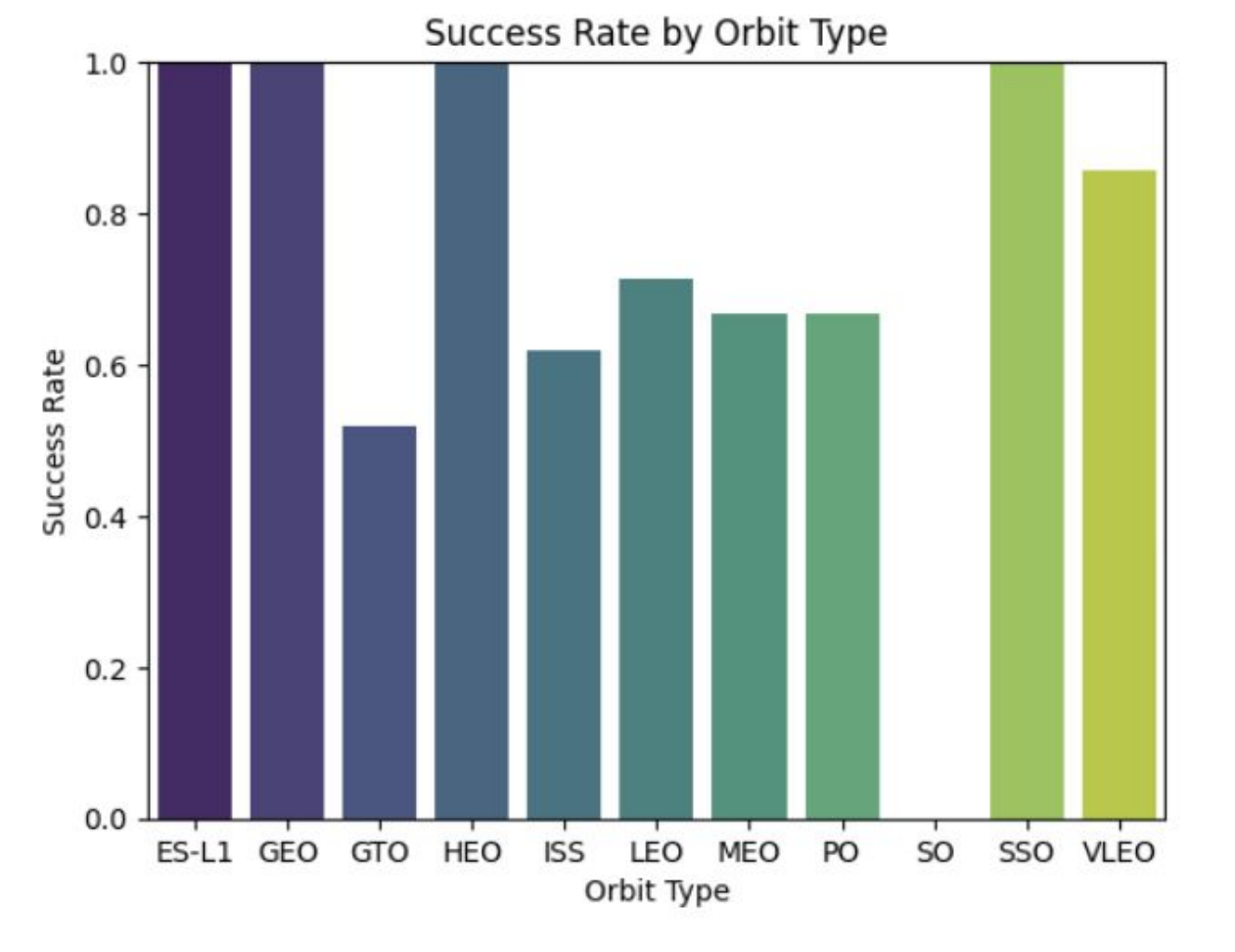
Payload vs. Launch Site



- Higher Flight Number indicates in general possible higher payloads

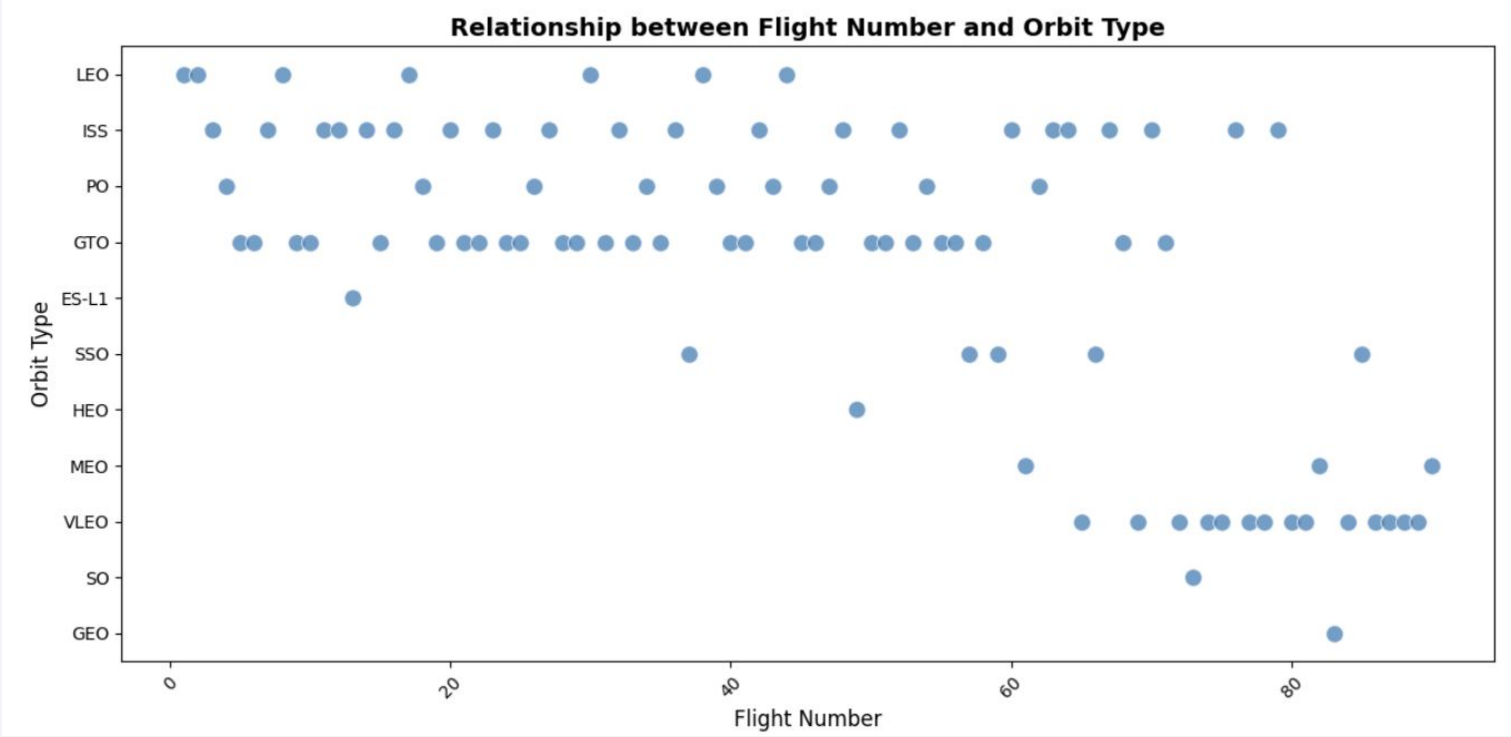
Success Rate vs. Orbit Type

- Oribts with 100% success rate:
  - ES-L1
  - GEO
  - HEO
  - SSO
  -



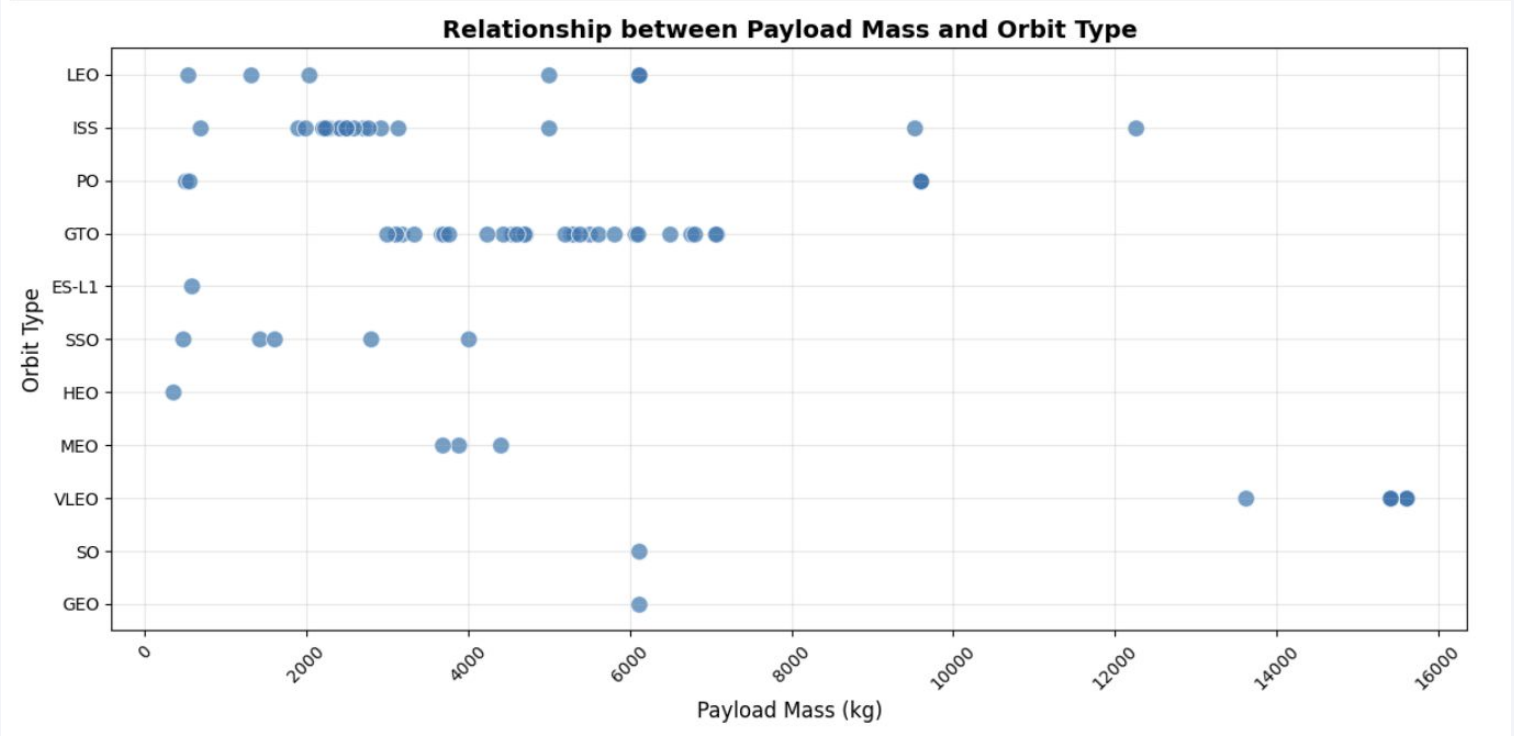
Flight Number vs. Orbit Type

- LEO/ ISS/ PO/ GTO/ ES-L1 beginning with lower Flight Numbers
- GEO/ SO/ VLEO/ MEO/ HE starting with Flight Numbers > #35
-



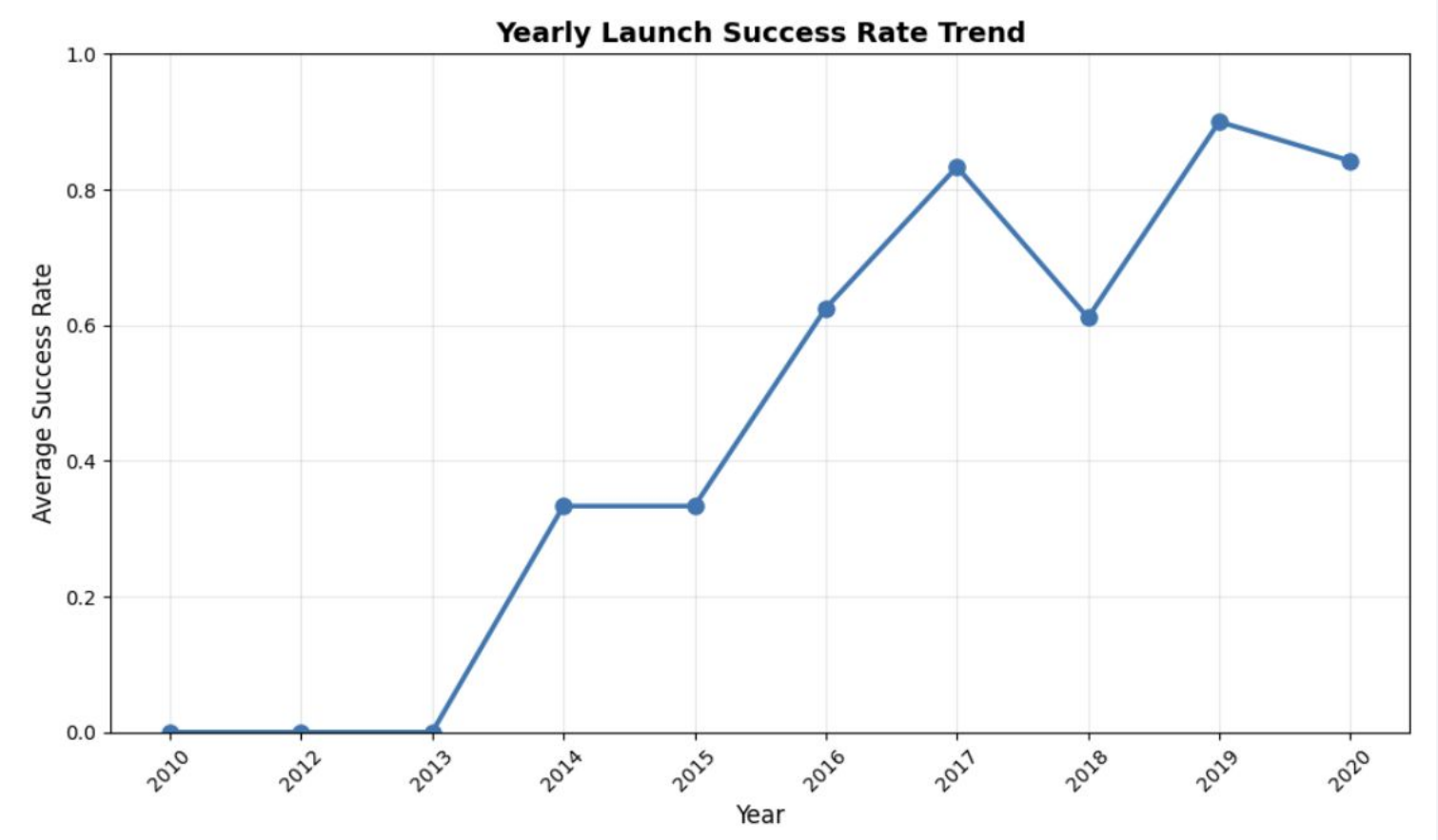
Payload vs. Orbit Type

- >8000kg Payload Mass only PO/ ISS/ VLEO Orbits are the destinations for SpaceX



Launch Success Yearly Trend

- more years of launching SpaceX means a higher average success rate



All Launch Site Names

Space X launch facilities: Cape Canaveral Space Launch Complex 40 **VAFB SLC 4E** , Vandenberg Air Force Base Space Launch Complex 4E **(SLC-4E)**, Kennedy Space Center Launch Complex 39A **KSC LC 39A** .The location of each Launch Is placed in the column

	Launch Site	Lat	Long
0	CCAFS LC-40	28.562302	-80.577356
1	CCAFS SLC-40	28.563197	-80.576820
2	KSC LC-39A	28.573255	-80.646895
3	VAFB SLC-4E	34.632834	-120.610745

Launch Site Names Begin with 'CCA'

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcome	Flights	GridFins	Reused	Legs	LandingPad	Block	ReusedCount	Serial	Longitude	Latitude	Class
0	1	2010-06-04	Falcon 9	6104.959412	LEO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0003	-80.577366	28.561857	0
1	2	2012-05-22	Falcon 9	525.000000	LEO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0005	-80.577366	28.561857	0
2	3	2013-03-01	Falcon 9	677.000000	ISS	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0007	-80.577366	28.561857	0
3	4	2013-09-29	Falcon 9	500.000000	PO	VAFB SLC 4E	False Ocean	1	False	False	False	NaN	1.0	0	B1003	-120.610829	34.632093	0
4	5	2013-12-03	Falcon 9	3170.000000	GTO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B1004	-80.577366	28.561857	0
5	6	2014-01-06	Falcon 9	3325.000000	GTO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B1005	-80.577366	28.561857	0

- Flight Numbers 0/1/2/4/5 starts with the Names ‘CCA’

```
%sql select distinct"Launch_Site" from SPACEXTABLE where "Launch_Site" like "CCA%"
```

```
 * sqlite:///my_data1.db
Done.
```

Launch_Site
CCAFS LC-40
CCAFS SLC-40

Total Payload Mass

```
%sql select sum("PAYLOAD_MASS__KG_") from SPACEXTABLE
 * sqlite:///my_data1.db
Done.
```

sum("PAYLOAD_MASS__KG_")
619967

- Total Payload Mass Space X = 619967 kg

Average Payload Mass by F9 v1.1

- average payload mass carried by booster version F9 v1.1:

avg("PAYLOAD_MASS__KG_"): 2534.6666666666665kg acc. the database

First Successful Ground Landing Date

- Find the dates of the first successful landing outcome on ground pad
- First successful Landing on ground pad at the 22th December 2015

```
%sql select "Landing_Outcome", min("Date") from SPACEXTABLE where "Landing_Outcome" like "Success (ground pad)"
```

```
 * sqlite:///my_data1.db
Done.
```

Landing_Outcome	min("Date")
Success (ground pad)	2015-12-22

Successful Drone Ship Landing with Payload between 4000 and 6000

```
%sql select "Booster_Version", "Landing_Outcome", "PAYLOAD_MASS__KG_" from SPACEXTABLE where "Landing_Outcome" like "Success (drone ship)" and PAYLOAD_MASS__KG_ between 4000 and 6000
```

```
 * sqlite:///my_data1.db
Done.
```

Booster_Version	Landing_Outcome	PAYLOAD_MASS__KG_
F9 FT B1022	Success (drone ship)	4696
F9 FT B1026	Success (drone ship)	4600
F9 FT B1021.2	Success (drone ship)	5300
F9 FT B1031.2	Success (drone ship)	5200

- Booster Version for successful Droneship landing

Total Number of Successful and Failure Mission Outcomes

- SQL Querry Mission outcomes

Done.

successful	failure
100	1

Boosters Carried Maximum Payload

- Boosters with max. Payload and Customer

Booster_Version	PAYLOAD_MASS__KG_	Customer
F9 B5 B1048.4	15600	SpaceX
F9 B5 B1049.4	15600	SpaceX
F9 B5 B1051.3	15600	SpaceX
F9 B5 B1056.4	15600	SpaceX
F9 B5 B1048.5	15600	SpaceX
F9 B5 B1051.4	15600	SpaceX
F9 B5 B1049.5	15600	SpaceX, Planet Labs
F9 B5 B1060.2	15600	SpaceX
F9 B5 B1058.3	15600	SpaceX
F9 B5 B1051.6	15600	SpaceX
F9 B5 B1060.3	15600	SpaceX
F9 B5 B1049.7	15600	SpaceX

2015 Launch Records

- 3 failed landing outcomes on droneship

Year-Month	Booster_Version	Launch_Site	Landing_Outcome
2015-01	F9 v1.1 B1012	CCAFS LC-40	Failure (drone ship)
2015-04	F9 v1.1 B1015	CCAFS LC-40	Failure (drone ship)
2015-06	F9 v1.1 B1018	CCAFS LC-40	Precluded (drone ship)

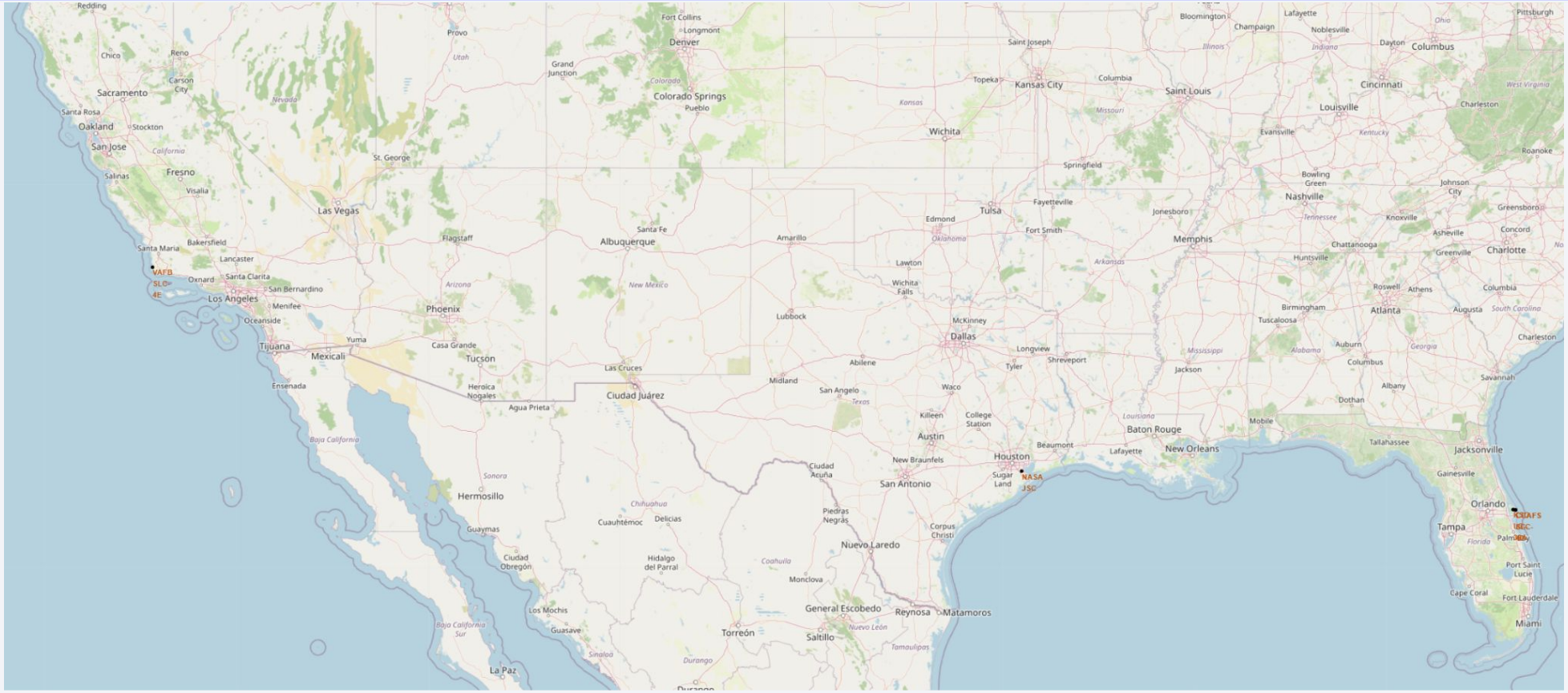
Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

Landing outcomes ranking descending from highest to lowest

Landing_Outcome	count_landings
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1


Section 3
Launch Sites
Proximities Analysis

USA SPACEX LAUNCH SIDES OVERVIEW



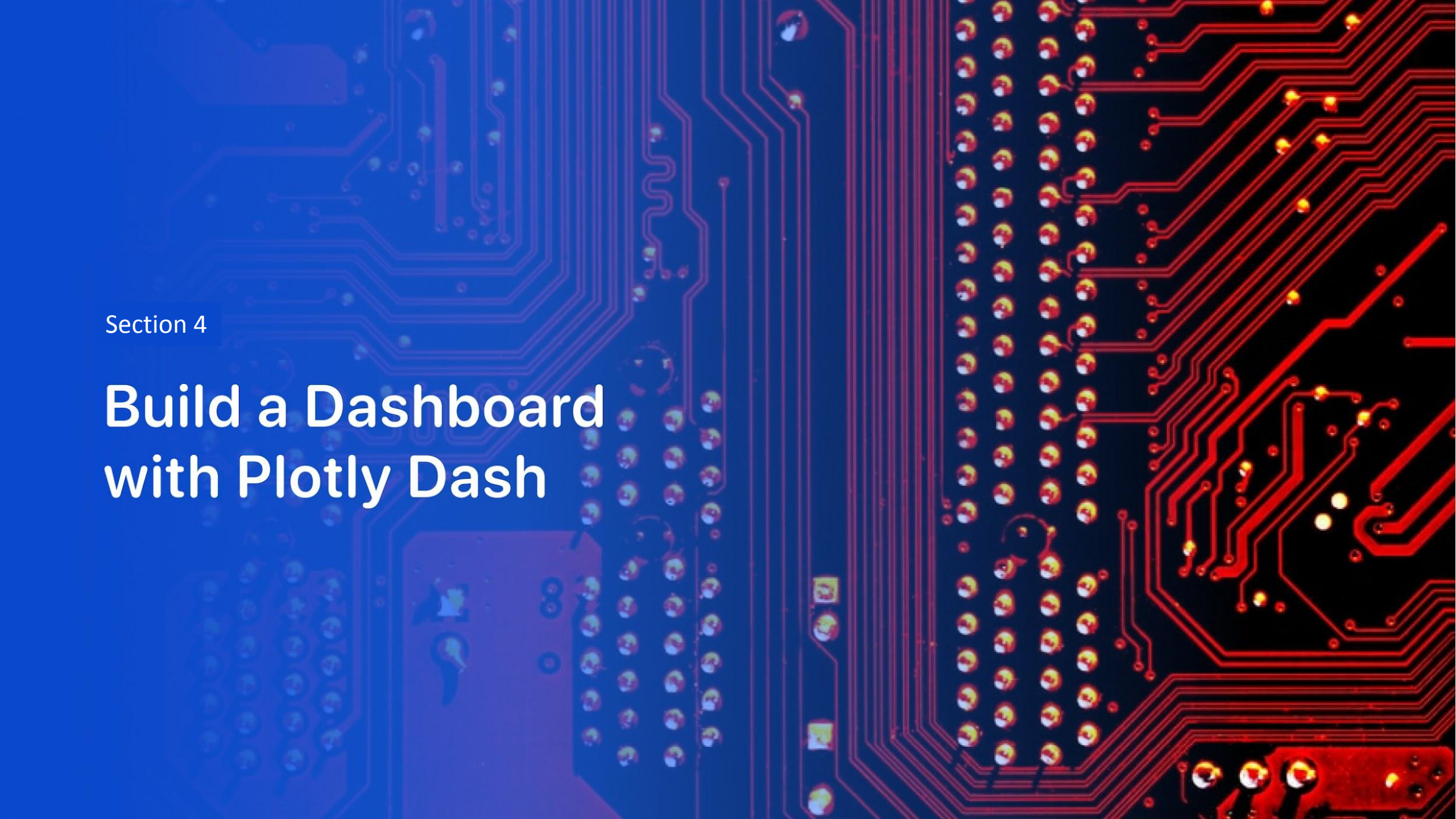
All launch sides are in the proximity of the ocean and the equator - launch sides are not in the proximity of cities/ railways/ highways/ highly populated areas

Success/ Fail status on Launch Sides

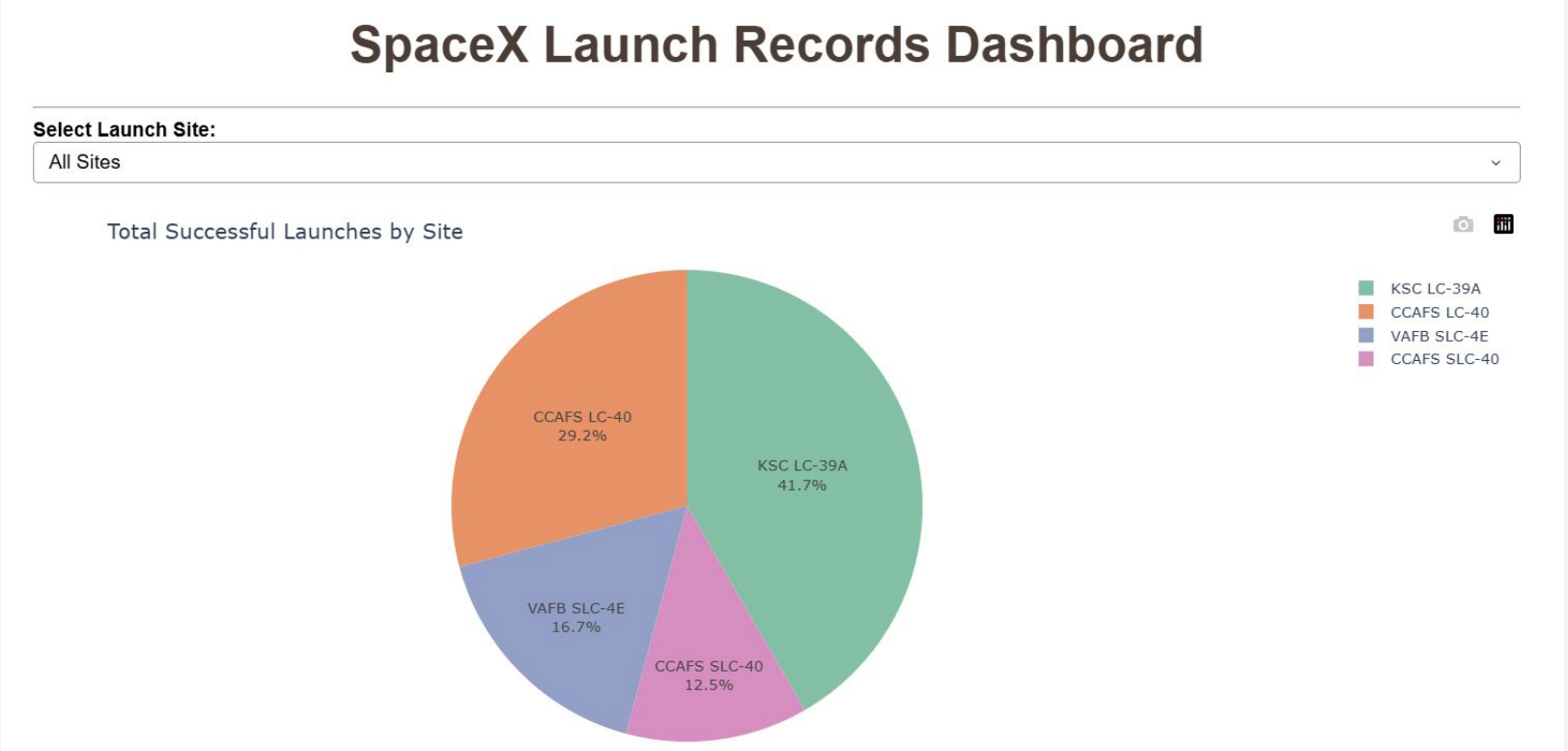
Green markers = success - Red markers = failed

Proximity Map for Launch Side

- Distance from the ocean

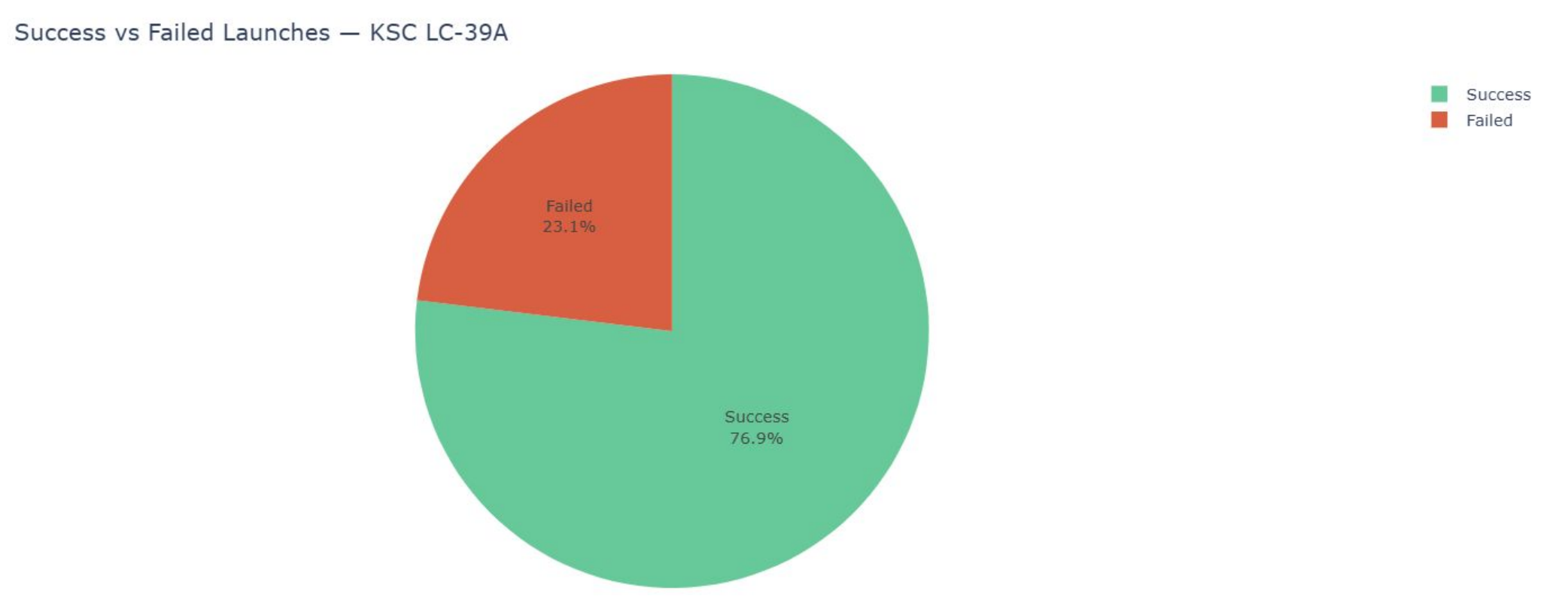

Section 4
Build a Dashboard with Plotly Dash

Launch Records Dashboard



CCAFS LC-40 (höchste absolute Success-Count im Pie Chart)

Score Success/ Fail



KSC LC-39A (höchstes Verhältnis Success/(Success+Fail))

Payload Mass vs. Launch Outcome

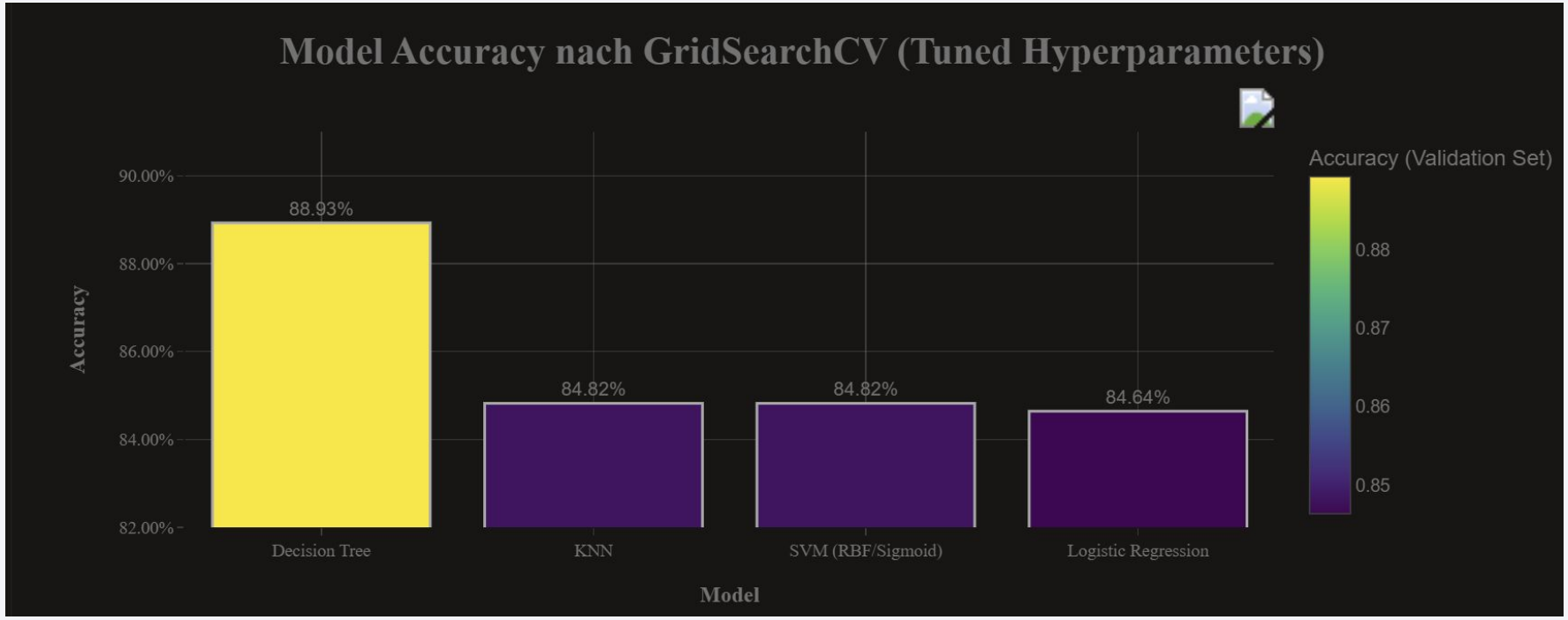
~5000–6000 kg (meiste(class=1)-Punkte im Scatter)

>9000 kg (meist(class=0)-Punkte bei hohen Payloads)

Section 5

Predictive Analysis (Classification)

Classification Accuracy

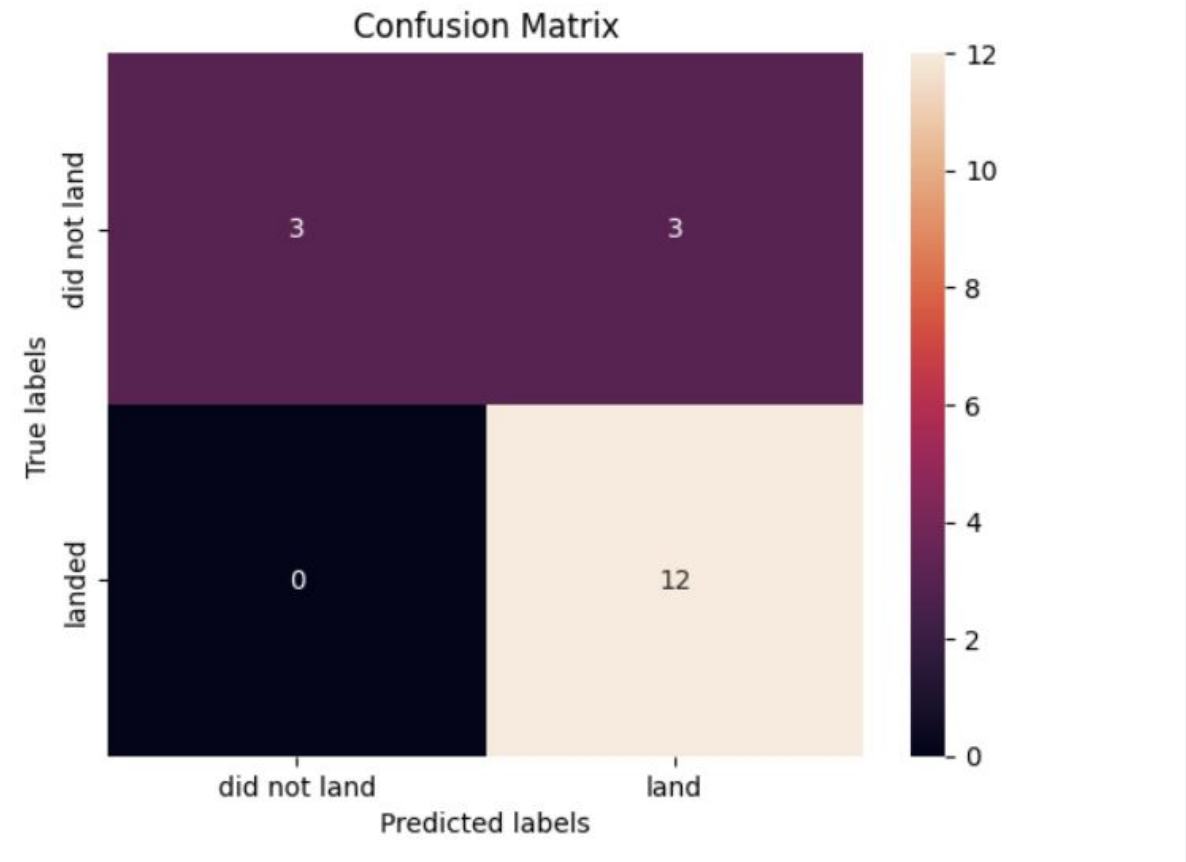


The Model with the best accuracy is the DecisionTree Model

Confusion Matrix - DecisionTree

- The accuracy with 0,889 from decisionTree is better then the accuracy from the other algorithms

```
yhat = tree_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)
```



Conclusions

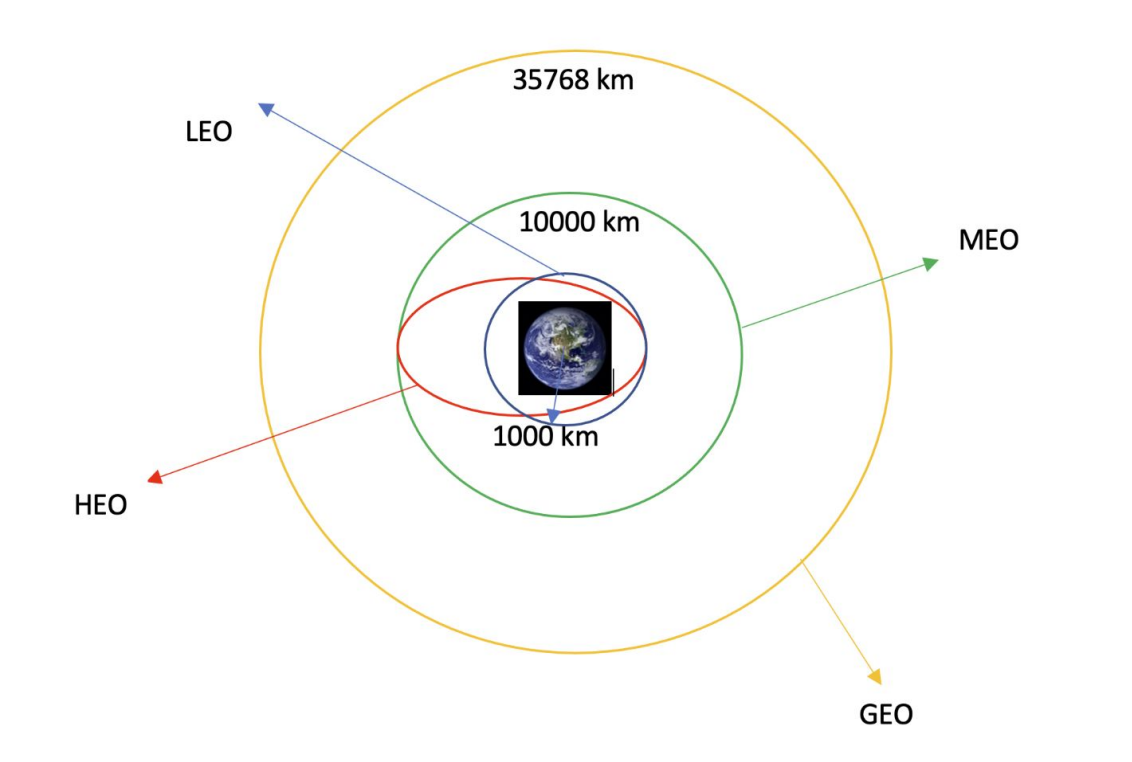
- DecisionTree is the best fitting Model with more than 4,2%points better than the next model
- All other models are in the very close range 84,7% accuracy
- Hyperparameter-Tuning works best with DecisionTree
- RandomForest maybe better than DecisionTree

Appendix

- Include any relevant assets like Python code snippets, SQL queries, charts, Notebook outputs, or data sets that you may have created during this project

Appendix

- Orbit definition



- **LEO**: Low Earth orbit (LEO)is an Earth-centred orbit with an altitude of 2,000 km (1,200 mi) or less (approximately one-third of the radius of Earth),[1] or with at least 11.25 periods per day (an orbital period of 128 minutes or less) and an eccentricity less than 0.25.[2] Most of the manmade objects in outer space are in LEO [1].
- **VLEO**: Very Low Earth Orbits (VLEO) can be defined as the orbits with a mean altitude below 450 km. Operating in these orbits can provide a number of benefits to Earth observation spacecraft as the spacecraft operates closer to the observation[2].
- **GTO**(Geostationary Transfer Orbit): A geostationary transfer orbit is an elliptical Earth orbit used to transfer satellites from low Earth orbit (LEO) to geostationary orbit (GEO). In a GTO, the perigee (closest point to Earth) is much lower than GEO altitude, while the apogee (farthest point) reaches approximately 22,236 miles (35,786 kilometers) above Earth's equator — the altitude of a geostationary orbit. Satellites in GTO use onboard propulsion to circularize their orbit at GEO altitude, where they can provide services such as weather monitoring, communications, and surveillance. [3] .
- **SSO (or SO)**: It is a Sun-synchronous orbit also called a heliosynchronous orbit is a nearly polar orbit around a planet, in which the satellite passes over any given point of the planet's surface at the same local mean solar time [4] .
- **ES-L1** :At the Lagrange points the gravitational forces of the two large bodies cancel out in such a way that a small object placed in orbit there is in equilibrium relative to the center of mass of the large bodies. L1 is one such point between the sun and the earth [5] .
- **HEO** A highly elliptical orbit, is an elliptic orbit with high eccentricity, usually referring to one around Earth [6].
- **ISS** A modular space station (habitable artificial satellite) in low Earth orbit. It is a multinational collaborative project between five participating space agencies: NASA (United States), Roscosmos (Russia), JAXA (Japan), ESA (Europe), and CSA (Canada) [7]
- **MEO** Geocentric orbits ranging in altitude from 2,000 km (1,200 mi) to just below geosynchronous orbit at 35,786 kilometers (22,236 mi). Also known as an intermediate circular orbit. These are "most commonly at 20,200 kilometers (12,600 mi), or 20,650 kilometers (12,830 mi), with an orbital period of 12 hours [8]
- **HEO** Geocentric orbits above the altitude of geosynchronous orbit (35,786 km or 22,236 mi) [9]
- **GEO** It is a circular geosynchronous orbit 35,786 kilometres (22,236 miles) above Earth's equator and following the direction of Earth's rotation [10]
- **PO** It is one type of satellites in which a satellite passes above or nearly above both poles of the body being orbited (usually a planet such as the Earth [11]

Thank you!